Agentic Supply Chain Forecasting System Using LangGraph

Abstract

This project presents an **agent-based supply chain forecasting system** that combines statistical modeling with intelligent reasoning. The system is built using **LangGraph** and **LangChain**, leveraging modular agents to simulate real-world decision-making.

Unlike traditional forecasting systems that rely purely on numerical models, this approach integrates **qualitative signals (market trends)** with **quantitative data (historical sales)** to produce more adaptive and interpretable decisions.

Introduction

Supply chain management is a critical domain where accurate demand forecasting directly impacts:

- inventory costs
- customer satisfaction
- operational efficiency

Traditional forecasting techniques:

- assume stable demand patterns
- fail under dynamic market conditions

This project introduces an **agentic architecture**, where:

- each component behaves like an independent decision-maker
- decisions are combined through a structured workflow

Background Concepts

Agent-Based Systems

An agent:

- processes inputs
- applies logic
- produces outputs

Multiple agents together form a **distributed decision system**.

LangGraph

LangGraph allows:

- defining workflows as graphs
- nodes = agents
- edges = execution flow

This is ideal for:

- multi-step reasoning
- state-driven pipelines

System Architecture

State Design

The system uses a shared state:

```
class AgentState(TypedDict):
    sku: str
    inventory_level: int
    historical_sales: List[int]
    market_signal: str
    forecast_value: float
    final_decision: str
```

```
    messages: List[str]
```

Why this matters:

- Every agent **reads + updates same state**
- Ensures **data consistency**
- Enables **traceability**

Class Design

SupplyChainDB

```
class SupplyChainDB:
    def __init__(self):
        self.data = {}

    def add_record(self, sku, stock, sales):
        self.data[sku] = {"stock": stock, "sales": sales}

    def get_record(self, sku):
        return self.data.get(sku, {"stock": 0, "sales": [0]})
```

Explanation:

- Acts as **mock ERP system**
- Decouples data from logic
- Supports:
    - insert
    - retrieval

Algorithm

1. Compute average:

$$avg = (1/n) * \sum S_i$$

2. Detect market signal:

- If surge → boost demand

3. Apply adjustment:

$$forecast = average * \alpha$$

Where:

- $\alpha = 1.3$ (surge)
- $\alpha = 1.05$ (normal)

Executive Decision Logic

```
def executive_manager_node(state):
    if state["forecast_value"] > state["inventory_level"]:
        decision = "ORDER"
    else:
        decision = "HOLD"

    return {
        "final_decision": decision,
        "messages": ["Decision made"]
    }
```

Strategy:

- Prevent stockouts
- Avoid overstock

Code Flow

1. User input initialized
2. State created
3. Graph execution begins

Flow:

Input → Ingestion → Analyst → Forecast → Manager → Output

State Evolution Example

Step	State Change
Ingestion	adds stock + sales
Analyst	adds market signal
Forecast	adds forecast value
Manager	adds decision

Example Execution

Input:

Stock = 30

Sales = [10,12,15,13]

Process:

- Avg = 12.5
- Adjusted = 16.25

Output:

Decision = HOLD

Advantages

- Modular design
- Scalable
- Interpretable
- Combines qualitative + quantitative

Limitations

- Simple forecasting model
- No real-time data
- No persistence layer

Future Scope

- ML models (LSTM, ARIMA)

- API integration
- Dashboard (Streamlit)
- Multi-SKU processing

Conclusion

This project demonstrates how **agent-based architectures** can transform supply chain systems by:

- improving adaptability
- enabling reasoning
- structuring decision pipelines

References

- LangGraph Documentation
- LangChain Docs
- OpenAI API Docs